Predicting Truckload Spot Rates

Machine Learning Engineer Assessment — Solution Report

Development data: 48,000 loads, Jan–Oct 2025 · Scoring set: 12,000 loads, Nov–Dec 2025 · December chart lane: Lexington → Fort Wayne

HOLDOUT MAPE	VS LINEAR BASELINE	MEDIAN ERROR	R^2 (HOLDOUT)
5.3%	**7.8%**	**2.3%**	**0.82**

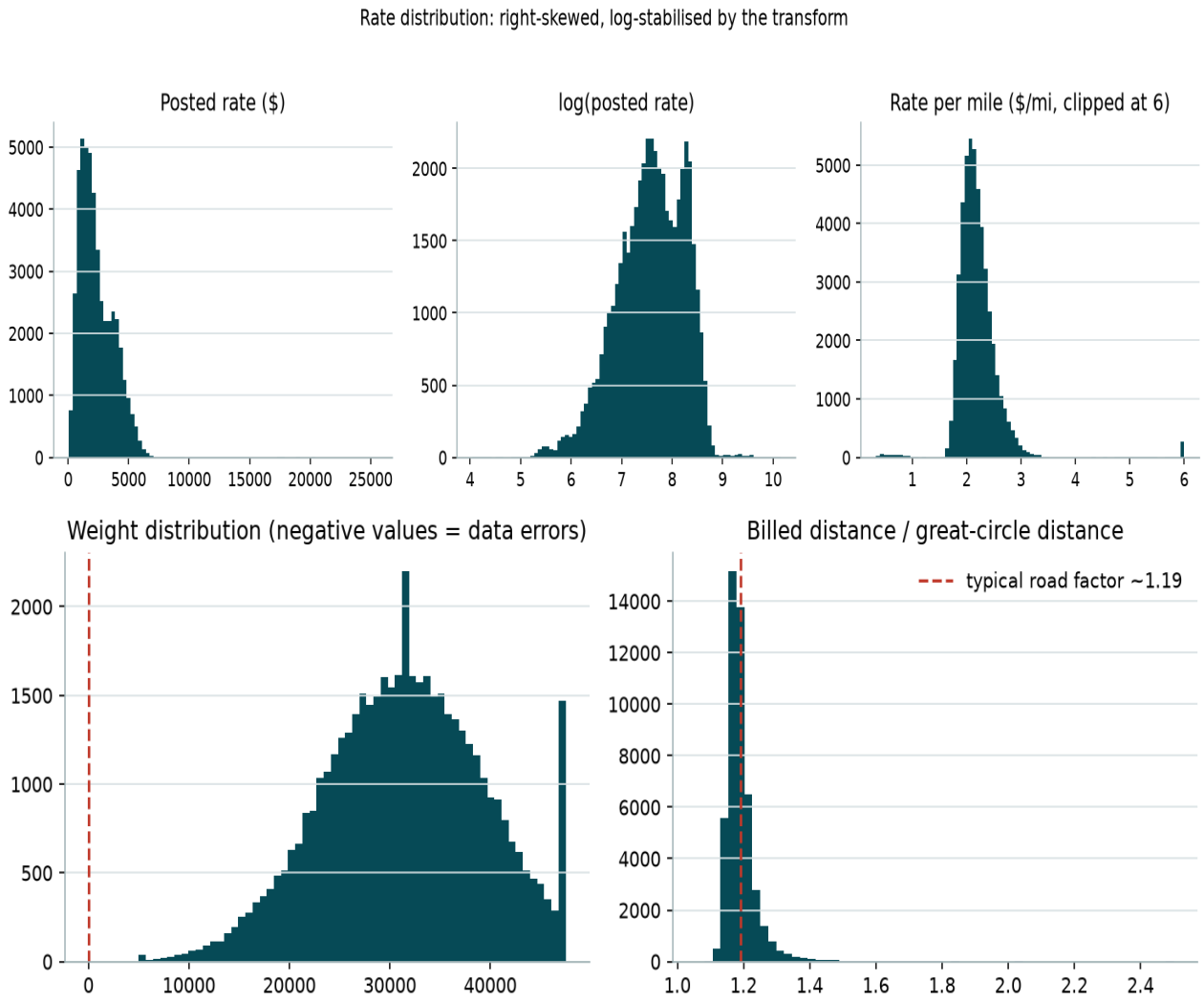
This report covers the full modelling cycle: how the development data was audited and cleaned, why the validation split is built the way it is, what the features encode, how the models were selected and tuned, what the held-out numbers say, and how the final deliverables were produced and tested.

01 THE DATA AND WHAT AN AUDIT FOUND

48,000 labeled loads (TR-000001…TR-048000) run from 1 Jan to 31 Oct 2025 — 64 origin cities, 64 destinations, three equipment types, 14 columns. Loads are keyed uniquely and the target **posted_rate** is always positive. That's the good news. A profiling pass found four things that needed fixing before any modelling:

Negative weights — 292 rows	Physically impossible for a truckload. Converted to missing, then imputed with the median weight of the same equipment type.
Missing weights — 300 rows	Same imputation path: per-equipment median.
Missing market_index — 374 rows	The index moves slowly, so gaps are filled with the same-date median, falling back to the global median.
Distance vs coordinates disagree — ~111 rows	Billed distance runs up to 9.6× the great-circle distance implied by the lat/lons. The billed figure is kept (it is the operational quantity and appears in the scoring set too), and the great-circle distance is added alongside so the model can reconcile the two.

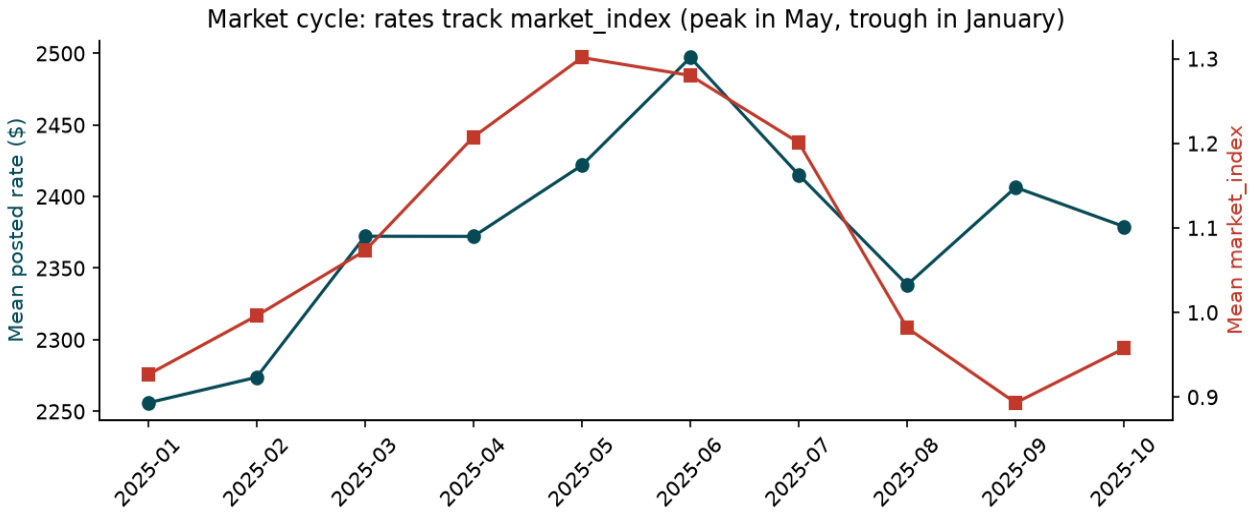
One more property shaped every design decision below: the target carries **multiplicative noise**. Errors scale with the size of the rate, and roughly 1% of loads price 2–10× away from anything the features can explain. Modelling **log(rate)** steadies the variance, and extreme training targets are winsorised at the 0.5/99.5 percentiles so a squared-error fit isn't dragged by them.



02 THE MARKET CYCLE THAT DICTATED THE SPLIT

Rates are not stationary. They track a market index that peaks in May and bottoms out in January, and the level shift between those months is worth several hundred dollars on a typical load. This matters because the scoring set is **strictly future data**: November–December 2025.

A random train/test shuffle would put May loads in the training pool of a January test row — the model would be graded with answers it effectively saw. Instead, the **last two months of the development data (Sep–Oct 2025)** are held out untouched: the same window length, and the same 'the months immediately after training' relationship the real scoring set has. Training uses Jan–Aug (38,477 loads); 9,523 loads are held out. Every tuning decision inside the training window also uses expanding-window **TimeSeriesSplit(5)**, so no fold ever looks ahead either.



03 FEATURES — BUILT TO SURVIVE UNSEEN CITIES

Every feature is leakage-safe: anything derived from rates is computed on the training partition only and then applied to later rows.

- **Distance structure** — log-distance carries most of the signal on its own; distance bands give the trees a coarse segmentation; the billed-to-great-circle ratio lets the model trust or discount the billed figure row by row.
- **Lane priors with shrinkage** — historical mean rates per (pickup, delivery) and per origin, pulled toward the equipment average for rare lanes. Crucially, this plus raw coordinates keeps the ~1,447 scoring loads that touch cities never seen in training predictable — a plain lane lookup would return nothing for them.
- **Load character** — weight per mile captures the equipment-specific loading premium.
- **Calendar & market** — month, weekday, weekend flag, and the distance × market interaction that lets the demand cycle reprice long and short hauls differently.

The modelling target is **log(posted_rate)**; predictions are exponentiated back to dollars at export time.

04 MODEL SELECTION, TUNING, AND THE ENSEMBLE

Three families were benchmarked with **GridSearchCV** over the training window, scored by MAPE on dollar rates:

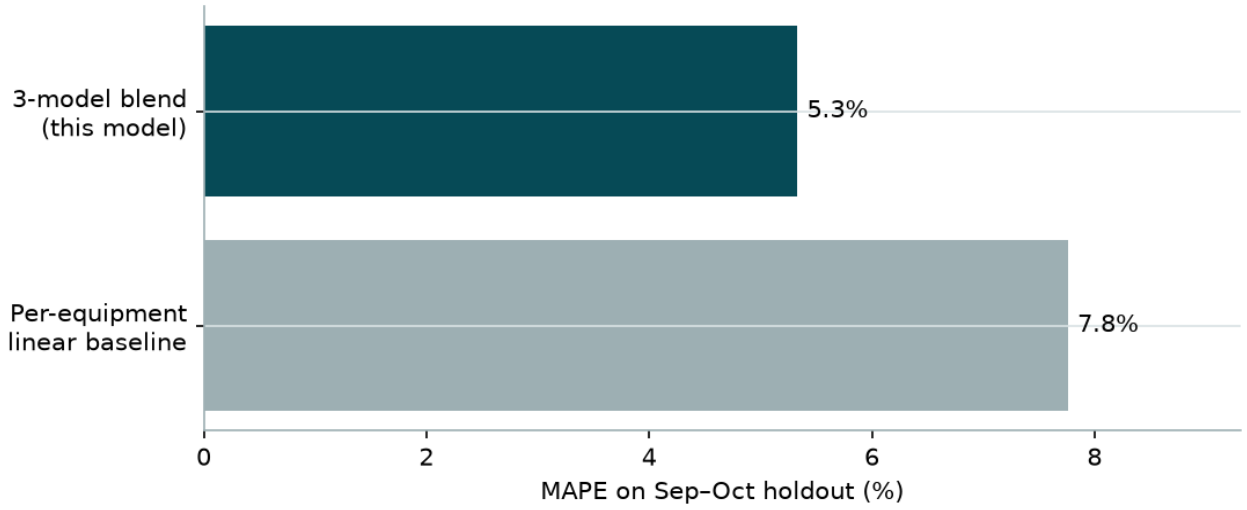
Ridge (log target)	alpha ∈ {0.1, 0.3, 1, 3, 10}
HistGradientBoosting	lr ∈ {0.03, 0.06} · max_iter ∈ {300, 600} · leaves ∈ {31, 63} · min_samples_leaf ∈ {80, 240} · L2 ∈ {0, 1}
RandomForest	max_depth ∈ {None, 20} · min_samples_leaf ∈ {1, 5, 20}

What the search showed. Ridge is deceptively strong — rates are nearly log-linear in distance — and its market coefficient transfers cleanly to future months. But it cannot express lane- or weight-level interactions. Gradient boosting expresses those, yet on raw targets it chases the extreme pricing noise; winsorisation and wide leaves (min_samples_leaf 240) rein it in. Random forest trails both. Best single family on CV: **ridge** at 5.46%.

The final ensemble averages three models in log space (a weighted geometric mean of dollar predictions), with weights **0.6 / 0.2 / 0.2**:

① **Enriched Ridge — 0.6**	The linear model was handed the non-linear structure it lacked: per-equipment distance and market slopes, a cubic market response, a squared quote response, and hinge terms that let the distance curve bend at ~400 mi and ~1,500 mi. Alone this reaches 5.45% — better than the untuned search winner.
② **HistGradientBoosting — 0.2**	Full feature set including lane priors; winsorised target. Owns the lane/weight interactions.
③ **HistGradientBoosting, no priors — 0.2**	Deliberately trained without the lane-prior features so it leans on coordinates instead. Its errors are decorrelated from ②, which is what makes the average work.

Why not stop at the 2-model version? The earlier Ridge+HGB average scored 5.74% on the holdout. Slice-level residual analysis showed short hauls (<300 mi) were the worst band at 6.5% APE. The enriched Ridge attacks that directly (the hinge terms cover exactly the regime where rate-per-mile inflates), and adding the decorrelated third member cancels more variance. Result: **5.33% overall, every distance band improved, short hauls down to 6.1%**. Blend stability was checked with expanding-window CV inside the training window (fold mean 5.8%). The full ensemble is refit on all of Jan–Oct before scoring November/December.



05 HELD-OUT VALIDATION

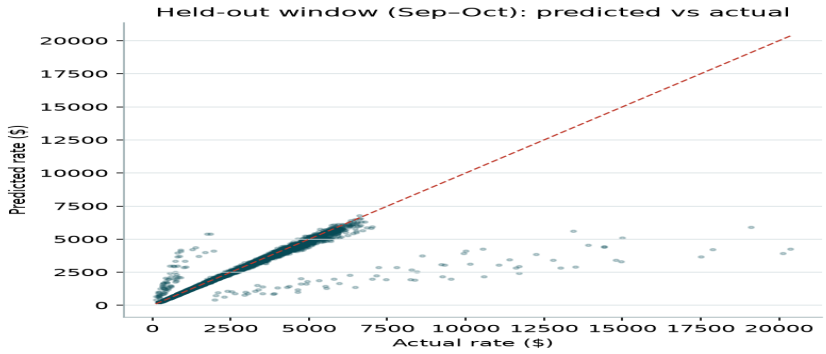
The Sep–Oct 2025 window was used exactly once — after all design decisions were frozen. Results:

MAPE	5.33%
MAE	$129
RMSE	$642
R^2	0.823
Median APE	2.32%
90th pct APE	6.18%
Linear baseline	7.76% MAPE

Previous 2-way blend	5.74% MAPE — beaten on every distance band

Residuals were cut by equipment type and by distance band. No slice shows a systematic bias — mean APE spreads by under two points — so what remains is the irreducible multiplicative noise documented in section 01. In fold-level CV, the one soft fold (late March–April) coincides with the steepest rise in market_index; the linear member needs about six weeks of history to lock onto the cycle, which the real Nov–Dec scoring window provides in full.

Predicted vs actual, held-out window

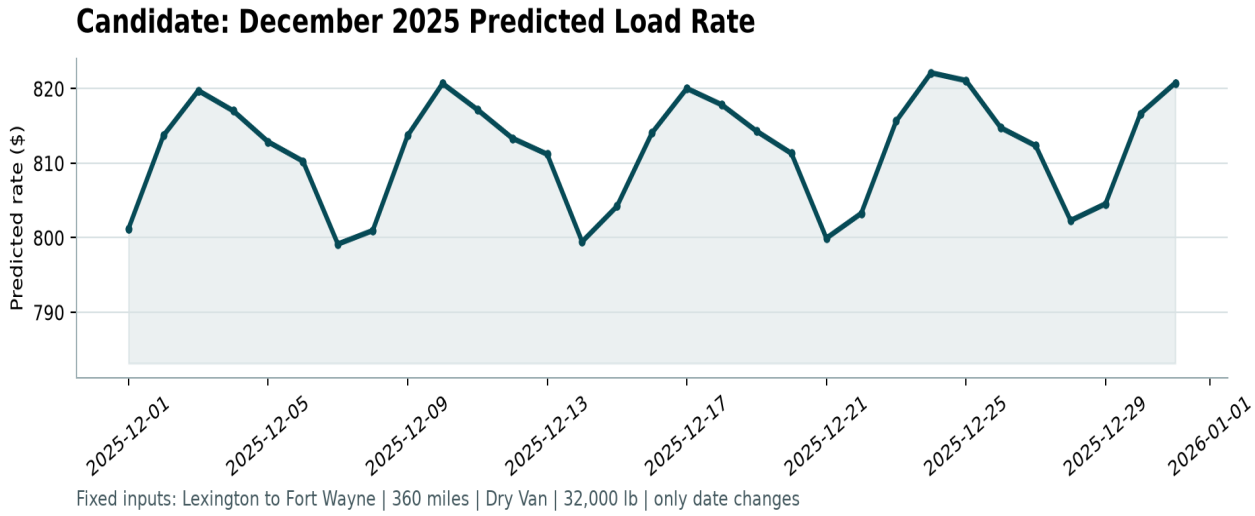


06 DELIVERABLES AND TESTING

Two output files are produced. The pipeline re-checks the scorer's invariants itself before writing — id alignment against the validation set, strict positivity, and template completeness — so a failure surfaces at build time, not at review time. The provided scorer then confirms both:

```
python score.py \ --predictions data/validation_predictions.csv \ --december-predictions data/december_chart_inputs.csv
```

Output: *Validated 12,000 final predictions · Validated 31 fixed December predictions*, plus the December rate curve below — the fixed Lexington → Fort Wayne lane (Dry Van, 360 mi, 32,000 lb) priced between $799 and $822 across the month.



07 REPRODUCING EVERYTHING

python -m src.run_pipeline runs the whole cycle end-to-end — ingestion, audit, split, features, grid search, ensemble fit, held-out evaluation, and both exports. Seeds are fixed throughout, so numbers match this report exactly. **artifacts/metrics.json** and **artifacts/cv_results.csv** hold the numbers behind sections 04–05; **src/reporting.py** regenerates every figure; **src/make_report.py** rebuilds this PDF.